

Transformers, the tech behind LLMs

3Blue

The first time I watched this, I thought I understood it. I didn't. The second time taught me what I'd missed. These are the notes from the third pass -- when the ideas finally felt simple.

§ 1 - The setup > 0:00

A transformer is a function that re-mixes vectors.

Forget the diagrams. The simplest mental model: you start with a sequence of vectors, and at each layer the model produces a new sequence of vectors of the same shape -- each one a re-mix of the others.

That's it. Everything else -- attention, MLPs, residual streams -- is machinery for deciding how the re-mix happens.

What goes in, what comes out:

A GPT takes a sequence of tokens (words, sub-words, punctuation) and produces a probability distribution over what token comes next. Internally, each token is represented as a high-dimensional vector, and the transformer layers update those vectors until the last one encodes enough context to predict the next word.

****NOTE - ARCHITECTURE**** A transformer block = one attention layer + one feed-forward layer, repeated N times. GPT-3 has 96 such blocks. Each block's job: gather information from across the sequence, then process it in place.

Why this framing matters:

Most explanations start with the attention equation and never escape it. The vector re-mixing view is more durable: it tells you what the network is computing (a contextual representation of each position) before you worry about how it computes it.

The three things that actually happen:

1. Embed -- project each token into a 12,288-dim space (GPT-3)
2. Transform -- 96 rounds of "look at your neighbors, then think"
3. Unembed -- project the final vector back to a score over 50,000 vocabulary tokens

§ 2 - Embeddings > 4:32

Tokens get coordinates.

The first thing the model does is look up each token in a table and turn it into a 12,288-dim vector (for GPT-3). The geometry of that space is where the model stores meaning.

The embedding matrix:

Think of it as a lookup table: 50,257 rows (vocabulary size), 12,288 columns (embedding dimension). Each token is a row index. The model learns this table from scratch -- no hand-crafting.

What emerges from training is a geometry where meaning is structure:

****NOTE - INTUITION**** -- "king ? man + woman $\leadsto$ queen" works because directions in this space encode relationships, not just identities. The difference vector between any two related concepts tends to point the same way.

Position encoding:

Attention has no built-in sense of order -- a sequence of vectors is just a set. To inject order, the model adds a positional vector to each token embedding. For GPT, these are learned (not the fixed sinusoidal ones from the original paper).

The positional embedding for position k is learned jointly with everything else. The model figures out what ordering

Why high dimensions matter:

12,288 dimensions sounds absurd. But high-dimensional spaces have a surprising property: you can pack exponentially many nearly-orthogonal vectors. This lets the embedding space hold thousands of independent "features" without interference -- a property called the superposition hypothesis.

§ 3 - Attention > 11:08

Each token asks every other token a question.

Attention is the mechanism that lets tokens talk to each other. Every position generates two things: a query ("what am I looking for?") and a key ("what do I offer?"). The dot product between a query and a key is the attention score.

The three matrices:

- * W_Q -- turns a token's vector into a query
- * W_K -- turns a token's vector into a key
- * W_V -- turns a token's vector into a value

Attention score for position i attending to j :

$$\text{score}(i,j) = \text{query}_i \cdot \text{key}_j / \sqrt{d_k}$$

Softmax across all j gives a probability distribution. Values are then weighted and summed.

****NOTE - CAUSAL MASKING**** -- GPT is **autoregressive**: it can only attend to earlier positions. Future positions are masked to ?? before softmax, so they contribute nothing. This is what makes it a **causal** transformer.

Multi-head attention:

Rather than one attention pattern, GPT-3 runs 96 heads in parallel at each layer. Each head uses lower-dimensional Q/K/V matrices (128-dim each), attends to the sequence, and produces its own output. The outputs are concatenated and linearly projected back.

Different heads specialize differently: some track syntactic dependencies, some track coreference, some do something that resists English description.

What attention is really doing:

The output of the attention step is a sum of value vectors, weighted by how relevant each position is. Concretely: if "it" strongly attends to "cat" in "the cat sat on the mat, and it was happy," then the representation of "it" gets mixed with the representation of "cat."

The mechanism doesn't store anything. It routes information from where it is to where it's needed.

§ 4 - Residual streams > 17:42

The bus, not the tape.

Every transformer block adds its output to its input -- it doesn't replace it. This creates a residual stream: a vector that flows through all 96 layers, with each block writing small corrections to it.

Why residuals matter:

Without residual connections, deep networks are hard to train (vanishing gradients). But there's a deeper reason to care: the residual stream is best understood as a communication bus that all components write to and read from.

The embedding layer writes the initial token vector onto the bus. Each attention head reads from the bus, computes its contribution, and writes back. Each MLP reads from the bus, runs its transformation, and writes back. The unembedding layer reads the final state.

****NOTE - ANALOGY**** -- Think of it less like a data tape (each block replaces what came before) and more like a whiteboard: each block can add to, erase, or correct what's written, but the history of additions is what accumulates.

What MLPs store:

After the attention step, each position runs through a two-layer feed-forward network independently (no cross-position mixing here). These are large -- 4x the embedding dimension in GPT-3 (49,152 neurons).

Recent interpretability work (Geva et al., 2021) suggests MLPs act as key-value memories: the first layer selects which "memories" are relevant; the second layer reads their stored values. This is how factual associations ("Paris is the capital of France") are believed to be stored.

Layer norms:

Before each sub-layer (attention and MLP), the residual stream is layer-normalized. This stabilizes the scale of activations without touching the direction, which is where the information lives.

§ 5 - Output > 22:15

Read the bus at the last position.

After 96 blocks, the residual stream at the last token position holds the model's entire prediction context. The unembedding step projects it back to vocabulary space.

Unembedding:

A linear layer maps the 12,288-dim vector to 50,257 logits -- one per vocabulary token. The unembedding matrix is often tied (shared weights) with the embedding matrix, cutting parameters in half.

Logits -> probabilities via softmax. The next token is then sampled from this distribution.

Temperature:

Dividing logits by a temperature T before softmax controls randomness:

- * $T \rightarrow 0$: always pick the top token (greedy decoding)
- * $T = 1$: standard sampling
- * $T > 1$: flatter distribution, more randomness

****NOTE - GREEDY VS SAMPLING**** -- Greedy decoding is deterministic but often dull. Sampling with $T \sim 0.7-0.9$ produces more varied, human-feeling text. Nucleus (top- p) sampling is another option: sample from the smallest set of tokens whose cumulative probability exceeds p .

The autoregressive loop:

At inference time, the model:

1. Embeds all tokens so far
2. Runs 96 transformer blocks
3. Reads logits at the last position
4. Samples one token
5. Appends it and repeats

This is generation -- the model is its own input. The context window (4,096 tokens for GPT-3) is the only hard limit on how far back it can look.

What the model has actually learned:

The weights are fixed after training. The model is a pure function: same input -> same computation every time. All its "knowledge" is in the 175 billion parameters -- the embedding table, the Q/K/V matrices, the MLP weights, the unembedding table. Every forward pass is just matrix multiplications and nonlinearities. The coherence is emergent.

the residual stream is a bus. every block just writes to it. i spent an hour on the attention equation and missed this.